

LAB 1

Setting up the IASYS Environment and Exploring an Autonomous System

Ricardo Severino

In this first laboratory you will prepare the two software environments used throughout IASYS and run a complete autonomous-system demonstration. You are not expected to understand ROS 2 commands or source code yet. The goal is to observe the system, interact with it, and begin identifying the Sense-Plan-Act chain that will be studied and implemented during the following laboratories.

GOALS

- Prepare an Ubuntu 24.04 virtual machine that will be used throughout the course.
- Install ROS 2 Jazzy as the native IASYS development environment.
- Install Docker and ADE and prepare the legacy Autoware.Auto reference environment.
- Run a small autonomous system and observe its behaviour in RViz2.
- Identify, at a conceptual level, sensing/state information, planning, control and actuation.
- Experiment with goals and failure situations without yet analysing the ROS 2 computation graph.

Two environments, two purposes

Native ROS 2 Jazzy (Ubuntu 24.04) is the environment in which you will develop your own IASYS components in C++17/rclcpp. Autoware.Auto remains available inside ADE as an industrial reference stack based on ROS 2 Foxy/Ubuntu 20.04. The two environments should be treated as separate. Python is used only for ROS 2 launch/configuration files and small auxiliary tools.

1 ENVIRONMENT OVERVIEW

IASYS deliberately uses two isolated software environments:

Environment	Purpose	Technology
IASYS development	Student code and all new laboratory developments	Ubuntu 24.04 + ROS 2 Jazzy + C++17/rclcpp
Industrial reference	Inspect and run selected Autoware.Auto examples	ADE/Docker + ROS 2 Foxy + Autoware.Auto

This separation allows the course to use a current ROS 2 distribution with a simplified model for actual student work while maintaining the Autoware.Auto autonomous driving stack as an industrial reference to be used throughout the course. [Autoware](#) is the world's first "all-in-one" open-source software for self-driving vehicles hosted under the Autoware Foundation. The [Autoware.Auto project](#), based on [ROS 2](#), is the next generation successor of the [Autoware.AI project](#), based on [ROS 1](#).

Check this LAB appendix for more information on these tools.

Throughout IASYS, functional ROS 2 nodes and autonomous-system algorithms will be implemented in C++17 using rclcpp and ament_cmake. Python is reserved for ROS 2 launch files and small auxiliary tools where appropriate.

2 SETTING UP THE DEVELOPMENT ENVIRONMENT

2.1 CREATE THE UBUNTU VIRTUAL MACHINE

Create a 64-bit Ubuntu 24.04 LTS virtual machine. The exact virtualization product is not important, but hardware virtualization must be enabled on the host computer.

- 4 virtual CPU cores or more
- 8 GB RAM minimum (12 GB recommended if available)
- 40 GB dynamically allocated virtual disk or more
- 3D acceleration enabled if your hypervisor supports it
- Network access enabled

Important

Autoware.Auto and its ADE images consume significant disk space. A 25 GB virtual disk, used in older IASYS instructions, is no longer recommended.

2.2 INSTALL ROS 2 JAZZY

Install ROS 2 Jazzy Desktop using the official [ROS 2 Ubuntu installation instructions](#). After installation, verify that the environment can be sourced:

```
source /opt/ros/jazzy/setup.bash
ros2 --help
rviz2
```

Install the development tools used in the course:

```
sudo apt update
sudo apt install -y python3-colcon-common-extensions python3-rosdep git
```

Initialise *rosdep* if this has not already been done:

```
sudo rosdep init
rosdep update
```

For now...

Do not explore the ros2 command-line interface. LAB 2 introduces nodes, topics, services, actions and the ROS 2 CLI systematically.

2.3 INSTALL DOCKER

Install the current Docker Engine for Ubuntu using [Docker's official repository instructions](#). After installation, configure your user so Docker can be used without sudo and log out/in before continuing.

```
$ sudo usermod -aG docker ${USER}
```

Check if it works.

```
docker --version
docker run --rm hello-world
```

Virtual machine note

Docker runs inside the Ubuntu virtual machine. This is expected: ADE will create Docker containers inside the VM. If Docker itself does not work correctly, do not proceed to ADE installation.

3 PREPARE THE AUTOWARE.AUTO REFERENCE ENVIRONMENT

Autoware.Auto is retained in IASYS as a reference industrial autonomous-driving stack. It is intentionally isolated from the native Jazzy environment.

3.1 INSTALL ADE

[ADE](#) creates a Docker container from a base image and mounts additional read-only volumes in /opt. The base image provides development tools (e.g. vim, udpreplay, etc.), and the volumes provide additional development tools (e.g., IDEs, large third-party libraries) or released software versions (e.g. the latest AutowareAuto release). Furthermore, ADE enables easy switching between versions of the images and volumes.

ADE requires a Linux host with Docker. Install the ADE executable and confirm that it is available in PATH. One possible procedure is:

```
wget https://gitlab.com/ApexAI/ade-cli/-
/jobs/1341322851/artifacts/raw/dist/ade+x86_64
mv ade+x86_64 ade
Give execute permissions.
chmod +x ade
sudo mv ade /usr/local/bin/ (you may need sudo here)
ade --version
# You should see 4.3.0
```

3.2 INSTALL AUTOWARE.AUTO

For ADE to function, it must be properly configured. Autoware.Auto provides an `.aderc` file which is expected to exist in the current working directory, or in any parent directory. Additionally, default configuration values can be overridden by setting environment variables. See the `ade --help` output for more information about using environment variables to define the configuration. Setup ADE by cloning Autoware.Auto. This will also be useful when you try to compile from source. You can find out more details about the installation process and the stack, at the [Autoware.Auto gitlab](https://gitlab.com/apexai/autowarefoundation/autoware.auto/AutowareAuto).

```
cd $(home)
mkdir adehome
cd adehome
touch .adehome
git clone https://gitlab.com/autowarefoundation/autoware.auto/AutowareAuto
cd AutowareAuto
```

Start and update ADE by entering this command

```
ade start --update
```

There are several preconfigured environments to choose from by specifying an ADE rc file. To see what is available, run:

```
$ ls -l .aderc*
```

Choose one, then launch with:

```
ade --rc .aderc-amd64-foxy start --update --enter
```

Congratulations! Now you should have a terminal inside ADE:

```
ade:~$
```

Upon entering, ADE outputs the images used to create the environment; e.g.

```
ade enter
Starting ade with the following images:
ade-foxy | 1641aaf554d9 | master |
registry.gitlab.com/autowarefoundation/autoware.auto/autowareauto/amd64/ade-foxy:master
binary-foxy | b5fb77ce958d | master |
registry.gitlab.com/autowarefoundation/autoware.auto/autowareauto/amd64/binary-foxy:master
```

Exit ADE when finished:

```
ade:~$ exit
ade stop #to stop the container
```

Note: Installation of system packages **does not persist** between `ade stop` and `ade start` commands. Anything valuable that needs to persist should be placed in the `~/adehome` directory, which is stored on the host and mounted in the ADE environment.

4 CHECK THE COMPLETE IASYS ENVIRONMENT AND BUILD

Copy the supplied LAB1 resources into your VM and run:

```
source /opt/ros/jazzy/setup.bash
cd <folder-containing-the-resources>
./scripts/check_environment.sh
```

Resolve any reported FAIL entries before continuing.

The demonstration package is intentionally small. In LAB1 you will use it as a black box. In later laboratories, several of its components will be replaced by your own implementations.

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/iasys_ws/src
cp -r <LAB1-resources>/iasys_demo ~/iasys_ws/src/
cd ~/iasys_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

Do not inspect the source code yet...

The objective for today is behavioural observation. We will open the black box gradually in LAB 2 and the following laboratories.

5 RUN AND EVALUATE THE AUTONOMOUS SYSTEM

```
cd ~/iasys_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch iasys_demo demo.launch.py
```

RViz2 should open automatically. The demonstration contains a 2-D world, static obstacles, a vehicle, a planned path and a goal. After startup the vehicle should plan a path around the obstacles and move autonomously toward the initial goal.

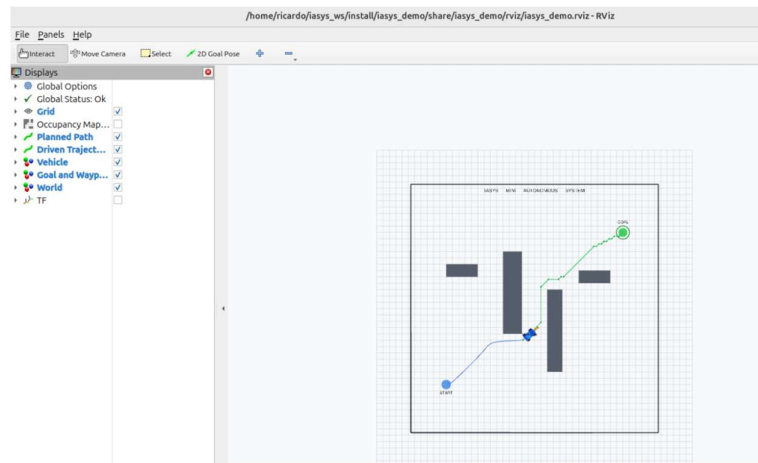


Figure 1 – Expected LAB1 view in RViz2: planned path (green), driven trajectory (blue), vehicle, start, goal and obstacles.

5.1 OBSERVE BEFORE CHANGING ANYTHING

Watch one complete run. Discuss the following questions with your group. Short answers are sufficient; no ROS terminology is required yet.

1. What information must the system know about the vehicle itself?
2. What information must it know about the environment?
3. Which part of the behaviour appears to correspond to planning?
4. Which part corresponds to actuation?
5. Which information appears to change continuously while the vehicle moves?
6. Does the vehicle appear to be following a pre-recorded motion, or reacting to a goal? Explain your reasoning.

5.2 CHANGE THE GOAL

In RViz2 select the **2D Goal Pose** tool and choose a new destination in a free region of the map. Repeat this several times.

1. Choose a goal on the other side of an obstacle.
2. Choose a goal close to the vehicle.
3. Choose a goal in another quadrant of the map.
4. Try to select a goal on top of an obstacle and observe the behaviour.

Record what changed when a new goal was selected. In particular, observe whether the path changes before the vehicle changes motion.

5.3 THINK IN SENSE-PLAN-ACT

Without using ROS 2 inspection tools, map the visible behaviour to the conceptual architecture discussed in the first lecture:

Environment/state information->sense->plan->act->vehicle

Complete the table based only on your observations:

Function	What you observed	What information is probably required
Sense / state acquisition		
Plan		
Act / control		

5.4 FAILURE EXPERIMENT

Close the demonstration and relaunch it with the controller disabled:

```
ros2 launch iasys_demo demo.launch.py controller_enabled:=false
```

Observe the system and answer:

- Does a valid path still exist?
- Does the vehicle move?
- What capability has been lost?
- Is the system still autonomous merely because some autonomous functions remain active?
- What should a safety-aware system do when an essential component fails?

Critical-systems perspective

Autonomy and dependability are different properties. A system can contain sophisticated autonomous functions and still be unsafe when information, computation or actuation fails.

6 WHAT COMES NEXT?

Today the system was deliberately treated as a black box. From LAB 2 onward, you will begin opening it and eventually replace several components yourself.

Stage	Capability	Your role
LAB 2	ROS 2 computation and communication	Inspect and create nodes, topics and services
	Perception	Process sensor data
	Localization	Estimate vehicle state
	Planning	Generate a path
	Control	Follow the path
	Cooperation	Exchange information and coordinate multiple vehicles

7 LAB 1 DELIVERABLES

Before leaving the laboratory, show the instructor:

- Ubuntu 24.04 VM running normally.
- ROS 2 Jazzy and RViz2 operational.
- Docker operational without sudo.
- ADE starts and the Autoware.Auto environment can be entered.
- IASYS demo builds and executes.
- Completed Sense–Plan–Act observation table and short answers to the failure experiment.

8 REFERENCES AND SUPPORT LINKS

ROS 2 Jazzy documentation: <https://docs.ros.org/en/jazzy/>

ADE installation: <https://ade-cli.readthedocs.io/en/latest/install.html>

Autoware.Auto installation with ADE:

<https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/installation-ade.html>

Autoware.Auto target environments:

<https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/target-environments.html>

APPENDIX

AUTOWARE.AUTO AND AUTOWARE.AI

In this Course we will rely upon the Autoware.Auto software stack to illustrate most of the course contents.

[Autoware](#) is the world's first "all-in-one" open-source software for self-driving vehicles hosted under the Autoware Foundation. The [Autoware.Auto project](#), based on [ROS 2](#), is the next generation successor of the [Autoware.AI](#), based on [ROS 1](#).

The major differentiators of Autoware.Auto compared to Autoware.AI were:

1. Modern software engineering best practices including code reviews, continuous integration testing, thorough documentation, thorough [test coverage](#), style and development guides.
2. Improved system architecture and module interface design (including messages and APIs).
3. An emphasis on reproducibility and determinism at the library, node, and system levels.

Autoware.Auto initially targeted the following two use cases:

- Autonomous Valet Parking
- Autonomous Cargo Delivery

As of January 2021, the valet parking functionality has been implemented and was showcased in a [live vehicle demonstration](#). It allows manoeuvring at low speeds on a parking lot.

The cargo delivery functionality was also placed under development under Autoware.Auto. The objective was to allow driving in a more difficult environment with intersections and indoor navigation, at moderate speeds. Each use case is designed with respect to a target **Operational Design Domain** (ODD), and the milestones for development can be viewed [here](#).

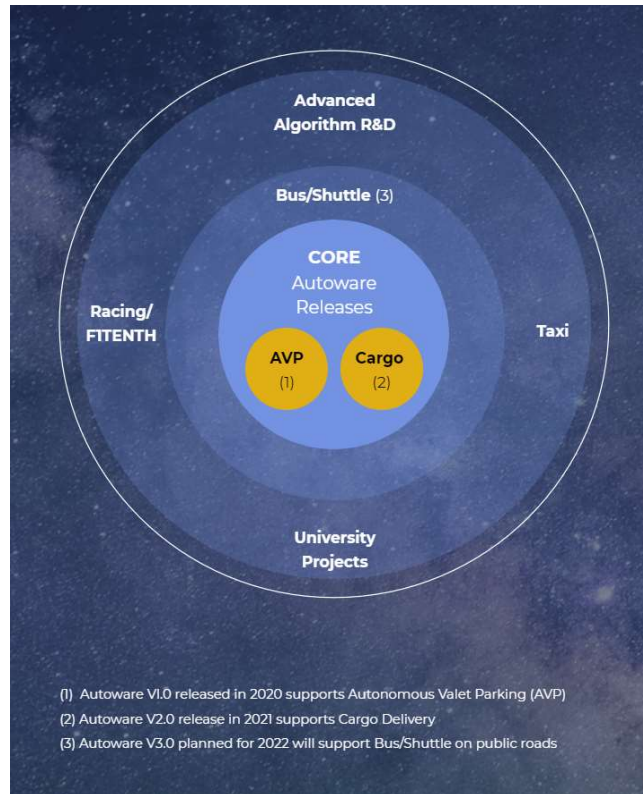
TOWARDS AUTOWARE CORE/UNIVERSE

Autoware.Auto project methodology proved its efficacy. However, the project was not entirely immune to a few drawbacks, particularly:

(1) the barrier to entry for the new engineers was too high, due to high coding and testing standards requiring substantial work to merge new features into the project, making it difficult for researchers and students to contribute to the development. As a consequence, most Autoware.Auto developers originated from member companies of the Autoware Foundation instead of a wide variety of contributors, making it extremely difficult to add state-of-the-art autonomous driving features from academic and scientific research to the project.

(2) Changes to the large-scale architecture became overwhelmingly cumbersome: When it was necessary to re-architect the stack, including new features or optimizing some components, there was a significant overhead involved in keeping the main branch stable while ensuring every change satisfied the continuous integration requirements.

To address the shortcomings in developing with Autoware.Auto, the Autoware Foundation decided to create a new architecture paradigm called **Autoware Core/Universe**.



Autoware Core/Universe paradigm can be visualized as a constellation. Autoware Core is at the core of the architecture. It carries over the original policy of Autoware.Auto to be a stable and well-tested codebase. Alongside Autoware Core is a new concept called Autoware Universe, which acts as an extension of Autoware Core.

Users can easily replace an Autoware Core component with an Autoware Universe equivalent to use more advanced features, such as a novel localization or perception algorithm.

Code quality requirements for Autoware Universe are more relaxed to make it easier for new developers, students and researchers to contribute, but still more strict than Autoware.AI

There is potential for inclusion of mature algorithms in the main Autoware Core codebase.

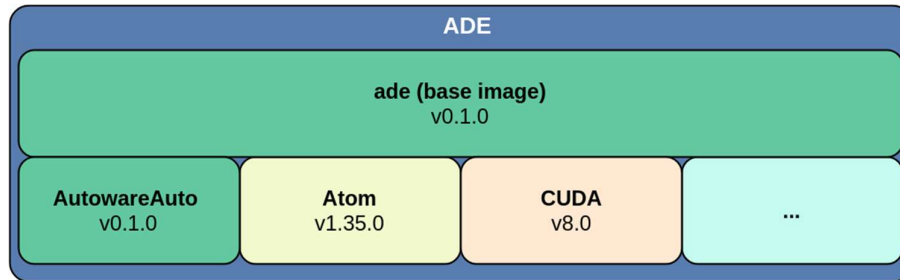
The ODDs under development are shown [here](#).

Despite the transition to Autoware 3.0, it is still worthwhile to work with the Autoware.Auto code. We will begin this first lab by installing the development environment you will be using throughout the course. *We recommend you install **Ubuntu 20.04 LTS** natively on your machine.* Dual booting can be a nice option. If you must, you can also setup a virtual machine environment. However, expect performance issues. We will need to also setup docker, ADE and finally Autoware.Auto.

ABOUT ADE “AWESOME DEVELOPMENT ENVIRONMENT”

[ADE](#) creates a Docker container from a base image and mounts additional read-only volumes in /opt. The base image provides development tools (e.g. vim, udpreplay, etc.), and the volumes provide additional development tools (e.g., IDEs, large third-party libraries) or released software

versions (e.g. the latest AutowareAuto release). Furthermore, ADE enables easy switching between versions of the images and volumes.



The image above shows an example of an ADE configuration. The base-image, ade, is used to create a container where volumes for AutowareAuto (a software library), Atom (an IDE), and CUDA (a large third-party library) are mounted as volumes.

ADE terminology:

Docker - Docker allows the creation of containers with isolated resources from the host OS: “Docker is a computer program that performs operating-system-level virtualization also known as containerization.” — [Wikipedia Docker \(software\)](#).

Image - Images are snapshots of the filesystem for a container. They are based on a Dockerfile, and are created using `docker build`. Images can be uploaded to a *registry*, where they can be downloaded by others.

Container - A container is a running instance of an image. When a container is deleted, any changes to its filesystem are usually discarded. A container can publish parts of its filesystem as *volumes*, which can then be mounted by other containers.

Registry - A registry is a server-side application that stores pre-built Docker images.

Fully-qualified name (FQN) - The FQN is the term for the full URL-like path that describes an image. e.g. `registry.gitlab.com/apexai/ade-atom:latest`

Tag - A tag is a specific version of an image. e.g. `latest` in the FQN above

Host - The host is the native operating system on which Docker is running.

When ADE is started, it prints out a corresponding matrix: e.g.

ade	v0.1.0	v0-1-0	registry.gitlab.com/autowareauto/autowareauto/ade:v0-1-0
autowareauto	v0.1.0	v0-1-0	registry.gitlab.com/autowareauto/autowareauto:v0-1-0
atom	v1.35.0	latest	registry.gitlab.com/apexai/ade-atom:latest
cuda	v8.0	v8-0	registry.private-gitlab/cuda/cuda:latest

The first row of the table is the base image, all other entries are volumes and have a corresponding entry in `/opt`. Each row displays the image name, version information (git hash or tag), the Docker tag, and the Fully-qualified name (FQN) for each image. If no version information was provided during the creation of the Docker image, the second column will be empty.

TIPS ON ADE USAGE AND CLEANING UP

When executing ade commands make sure you are in the ADE work directory:

`~/adehome/AutowareAuto`

ADE and the docker containers are now running. You can list the running Docker containers with:

```
docker ps # when ADE has running containers you can access then with:
ade enter # you can stop then with:
ade stop
```

To assess the disk usage situation, run the following command:

```
docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	2	2	6.119GB	0B (0%)
Containers	3	2	3.986MB	0B (0%)
Local Volumes	1	1	192.4MB	0B (0%)
Build Cache	0	0	0B	0B

To purge unused images:

```
docker system prune -a --volumes
```

Credits

Version 3.0, Ricardo Severino partially based on Autoware Training Materials